

Cucumber & Selenium

Shinta Nuraisya Arizky

Praktisi Mengajar 4 (PPPL) – Sekolah Vokasi UGM



Outline



01

**Cucumber
Introduction**

02

**Integrate
Cucumber with
Selenium**

03

**Best
Practice**

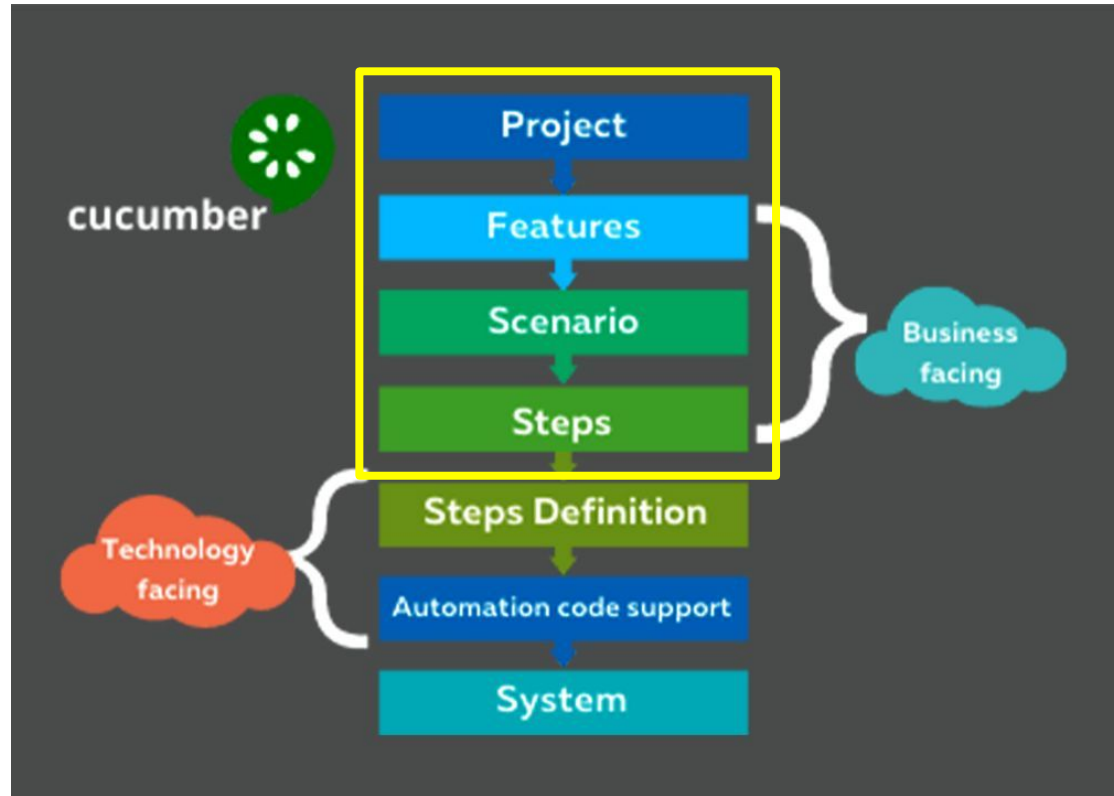


01

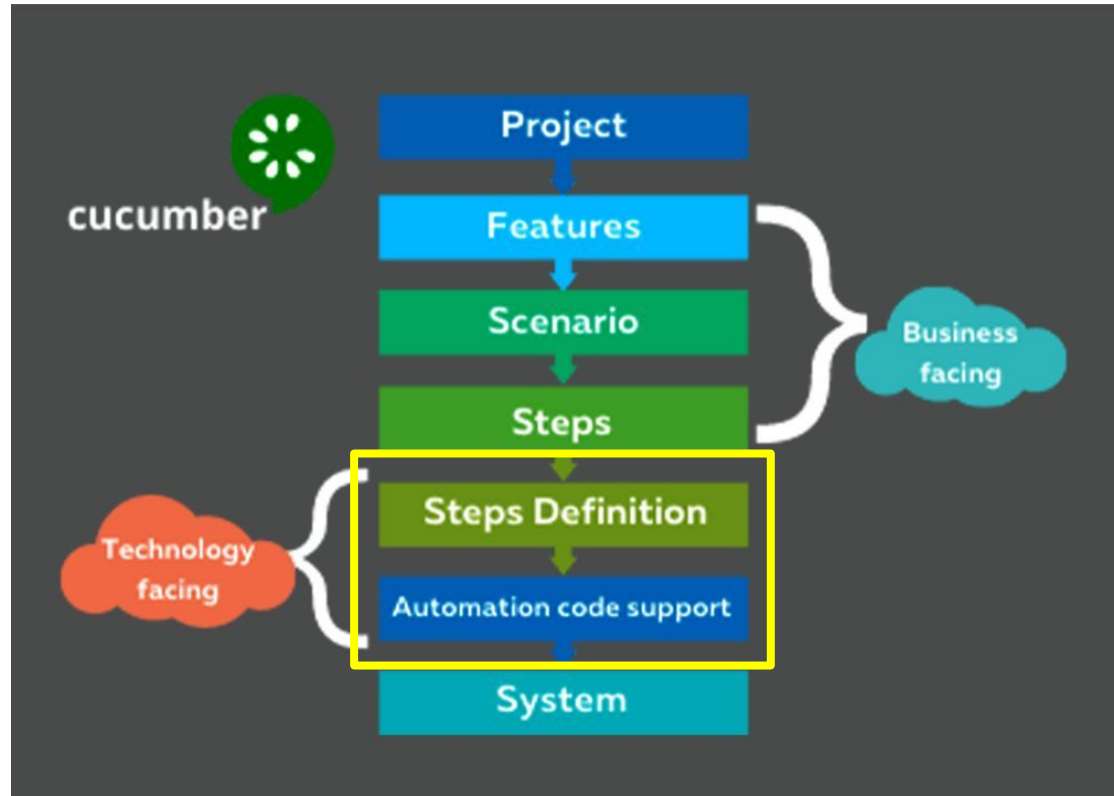
Cucumber Introduction



BDD



Cucumber



Cucumber

- **Open source** testing framework that supports Behavior Driven Development
- Can be implemented for **automation testing**
- In order for Cucumber to understand the scenarios, it must **follow some basic Gherkin syntax rules**



BDD Framework for Cucumber-Selenium

**BDD
Framework:**

01

Feature File

A standalone unit or a single functionality (such as a login) for a project

02

Step Definitions

Stores the mapping data between each step of a scenario defined in the feature file and the code to be executed

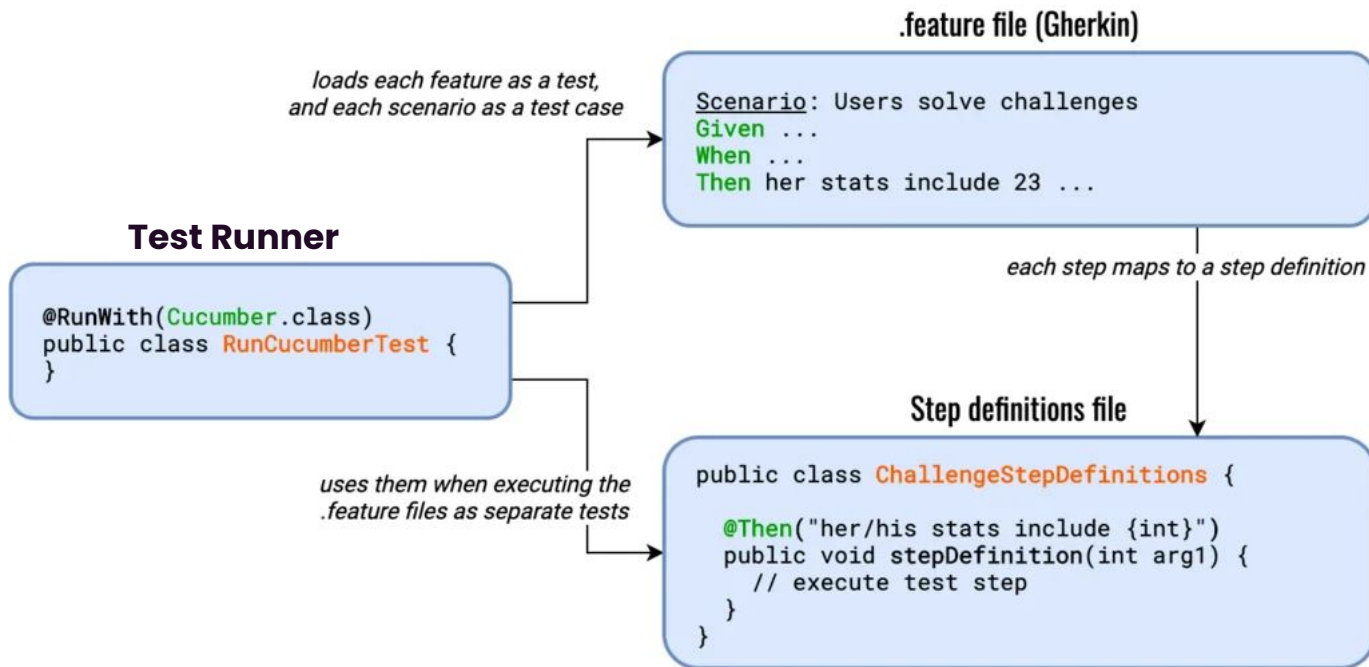
03

Test Runner

JUnit Test Runner Class containing the Step Definition location



BDD Framework for Selenium





02

Integrate Cucumber with Selenium



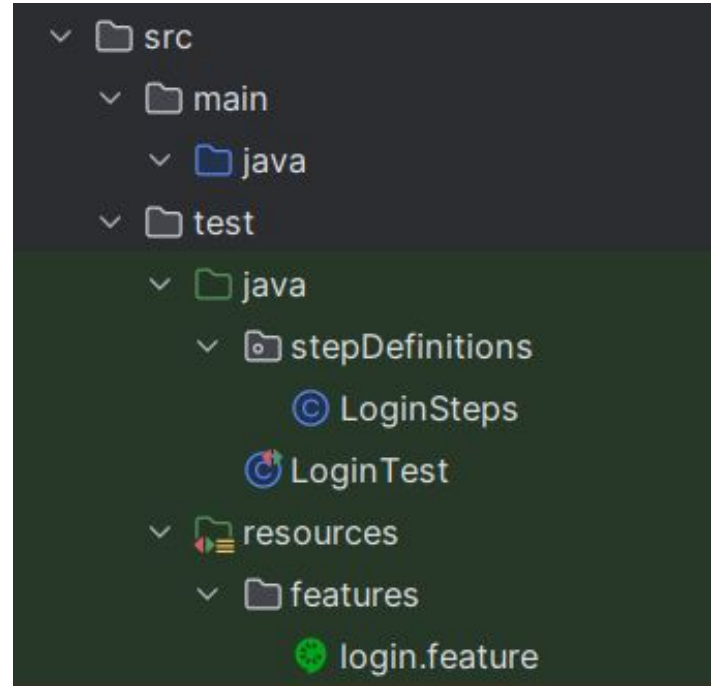
Add Cucumber Dependency

```
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.18.0</version>
</dependency>
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit</artifactId>
  <version>6.11.0</version>
  <scope>test</scope>
</dependency>
```



Integrate Cucumber with Selenium

1. Create feature file, e.g: Login.feature
2. Write test scenario using Gherkin syntax
3. Step Definitions
4. Setup Test Runner



1. Create Feature File – Login.feature

Feature: User Login

Scenario: Successful login with valid credentials

Given User is on the login page

When User submit valid credentials

Then User should be redirected to the dashboard

1. Create Feature File – Login.feature

```
Feature: User Login
```

```
Scenario: Successful login with valid credentials
```

```
    Given User is on the login page
```

```
    When User submit valid credentials
```

```
    Then User should be redirected to the dashboard
```

2. Step Definitions

```
public class LoginStepDef {
    WebDriver driver;

    void setupChromeDriver() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @Given("User is on the login page")
    public void navigateToLoginPage() {
        setupChromeDriver();
        driver.get("https://www.saucedemo.com/");
    }

    @When("User submit valid credentials")
    public void submitValidCredentials() throws InterruptedException {
        LoginPage login = new LoginPage(driver);
        login.enterUsername("standard_user");
        login.enterPassword("secret_sauce");
        login.clickLogin();
    }

    @Then("User should be redirected to the dashboard")
    public void loginSuccessful() throws InterruptedException {
        HomePage home = new HomePage(driver);
        home.getTextProducts();
        String expectedUrl = "https://www.saucedemo.com/inventory.html";
        Assert.assertEquals(home.getActualUrl(), expectedUrl);
    }

    @After()
    public void closeBrowser() {
        driver.quit();
    }
}
```

3. Test Runner

```
@RunWith(Cucumber.class)
@CucumberOptions(features = "src/test/resources/features",
glue = "stepDefinitions")

public class TestRunner {

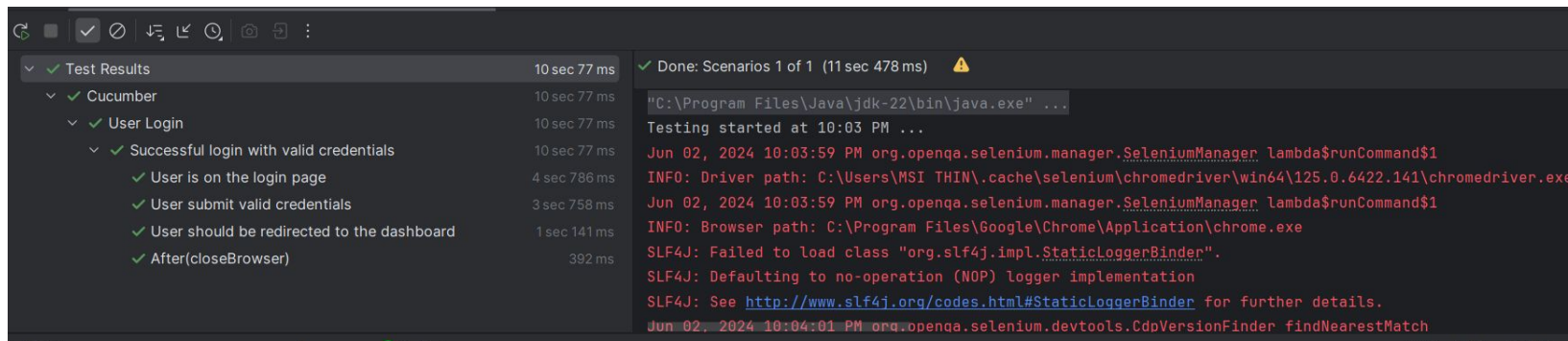
}
```

Run the Script

Run login.feature

Browser will be navigated to Saucedemo homepage and do login.

Wait until the script finished to run. If there is no error, the test result is **passed**.



The screenshot displays the Selenium IDE interface. The left pane shows the 'Test Results' for a scenario named 'login.feature'. The results are as follows:

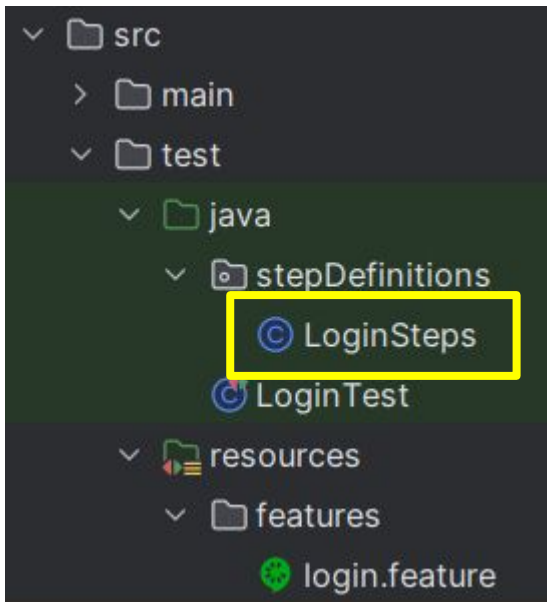
Test Step	Duration
Test Results	10 sec 77 ms
Cucumber	10 sec 77 ms
User Login	10 sec 77 ms
Successful login with valid credentials	10 sec 77 ms
User is on the login page	4 sec 786 ms
User submit valid credentials	3 sec 758 ms
User should be redirected to the dashboard	1 sec 141 ms
After(closeBrowser)	392 ms

The right pane shows the 'Command Log' for the scenario. It indicates that the scenario was completed successfully in 11 seconds and 478 milliseconds. The log includes the following information:

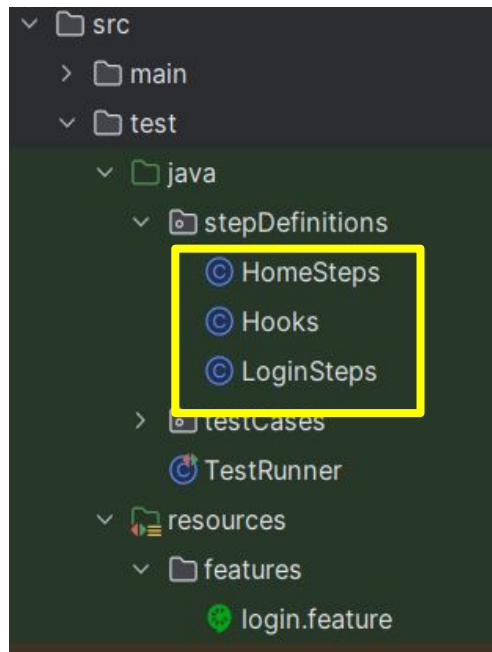
- Command: `"C:\Program Files\Java\jdk-22\bin\java.exe" ...`
- Testing started at 10:03 PM ...
- Jun 02, 2024 10:03:59 PM org.openqa.selenium.manager.SeleniumManager lambda\$runCommand\$1
- INFO: Driver path: C:\Users\MSI THIN\.cache\selenium\chromedriver\win64\125.0.6422.141\chromedriver.exe
- Jun 02, 2024 10:03:59 PM org.openqa.selenium.manager.SeleniumManager lambda\$runCommand\$1
- INFO: Browser path: C:\Program Files\Google\Chrome\Application\chrome.exe
- SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
- SLF4J: Defaulting to no-operation (NOP) logger implementation
- SLF4J: See <http://www.slf4j.org/codes.html#StaticLoggerBinder> for further details.
- Jun 02, 2024 10:04:01 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch

Enhanced Step Definitions

Enhanced Step Definitions



Before



After

Enhanced Step Definitions – Hooks

- Blocks of code that run before or after each scenario.
- Typically used for **setup** and **teardown** of the environment before and after each scenario.

```
@Before
public void setUp() {
    if (driver == null) {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }
}

@After
public void tearDown() {
    if (driver != null) {
        driver.quit();
        driver = null;
    }
}

public static WebDriver getDriver() {
    return driver;
}
```



Why Cucumber Hooks?

We need to perform the **prerequisite** steps before testing any test scenario.

- Starting a webdriver
- Setting up DB connections
- Setting up test data
- Navigating to certain page, etc

In the same way, there are always **after steps** as well of the tests like:

- Killing the webdriver
- Closing DB connections
- Clearing the test data
- Logging out from the application
- Printing reports or logs
- Taking screenshots on error, etc



LoginSteps

HomeSteps

```
public class LoginStepDef {
    WebDriver driver;

    void setupChromeDriver() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @Given("User is on the login page")
    public void navigateToLoginPage() {
        setupChromeDriver();
        driver.get("https://www.saucedemo.com/");
    }

    @When("User submit valid credentials")
    public void submitValidCredentials() throws InterruptedException {
        LoginPage login = new LoginPage(driver);
        login.enterUsername("standard_user");
        login.enterPassword("secret_sauce");
        login.clickLogin();
    }

    @Then("User should be redirected to the dashboard")
    public void loginSuccessful() throws InterruptedException {
        HomePage home = new HomePage(driver);
        home.getTextProducts();
        String expectedUrl = "https://www.saucedemo.com/inventory.html";
        Assert.assertEquals(home.getActualUrl(), expectedUrl);
    }

    @After()
    public void closeBrowser() {
        driver.quit();
    }
}
```

Hooks

Enhanced Step Definitions – LoginSteps

```
public class LoginSteps {
    WebDriver driver;
    LoginPage loginPage;

    public LoginSteps() {
        this.driver = Hooks.getDriver(); // Mendapatkan instance WebDriver dari Hooks
        this.loginPage = new LoginPage(driver);
    }

    @Given("User is on the login page")
    public void user_is_on_the_login_page() {
        driver.get("https://www.saucedemo.com/");
    }

    @When("User submit valid credentials")
    public void user_submit_valid_credentials() throws InterruptedException {
        loginPage.enterUsername("standard_user");
        loginPage.enterPassword("secret_sauce");
        loginPage.clickLogin();
    }
}
```

Enhanced Step Definitions – HomeSteps

```
public class HomeSteps {
    WebDriver driver;
    HomePage homePage;

    public HomeSteps() {
        this.driver = Hooks.getDriver(); // Mendapatkan instance WebDriver dari Hooks
        this.homePage = new HomePage(driver);
    }

    @Then("User should be redirected to the dashboard")
    public void user_should_be_redirected_to_the_dashboard() throws
    InterruptedException {

        //verify if the homepage contains text Products
        homePage.getTextProducts();

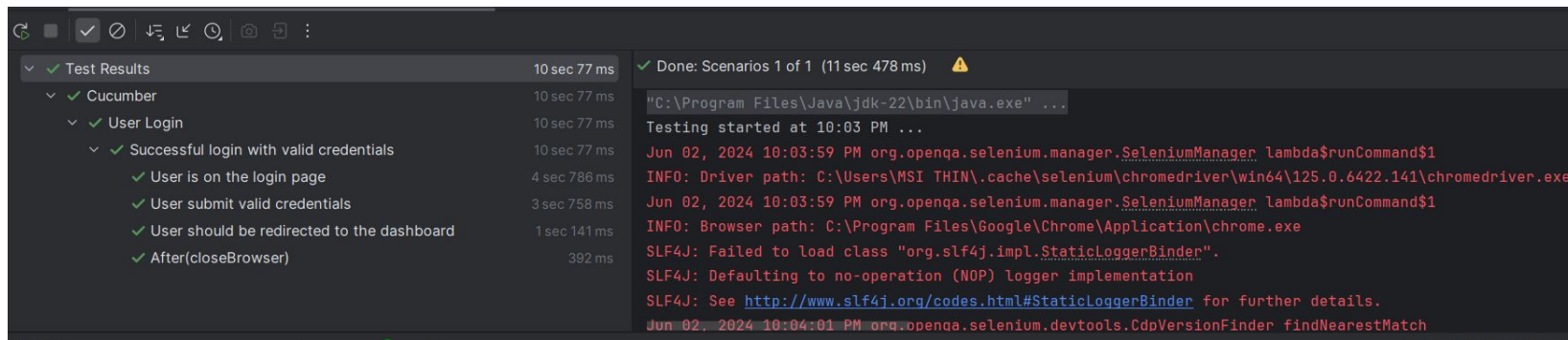
        //verify the current URL
        String expectedUrl = "https://www.saucedemo.com/inventory.html";
        Assert.assertEquals(homePage.getActualUrl(), expectedUrl);
    }
}
```

Run the Script

Run login.feature

Browser will be navigated to Saucedemo homepage and do login.

Wait until the script finished to run. If there is no error, the test result is **passed**.



The screenshot displays the Selenium IDE interface. On the left, the 'Test Results' pane shows a hierarchical view of the test execution. The 'User Login' scenario is expanded, showing four steps, all of which are marked with green checkmarks, indicating they passed successfully. The total duration for the 'User Login' scenario is 10 sec 77 ms. On the right, the 'Command Log' pane shows the execution details. It starts with 'Done: Scenarios 1 of 1 (11 sec 478 ms)' and a warning icon. Below this, it shows the command 'C:\Program Files\Java\jdk-22\bin\java.exe' and the start time 'Testing started at 10:03 PM ...'. The log then shows several log messages: 'Jun 02, 2024 10:03:59 PM org.openqa.selenium.manager.SeleniumManager.lambda\$runCommand\$1', 'INFO: Driver path: C:\Users\MSI THIN\.cache\selenium\chromedriver\win64\125.0.6422.141\chromedriver.exe', 'Jun 02, 2024 10:03:59 PM org.openqa.selenium.manager.SeleniumManager.lambda\$runCommand\$1', 'INFO: Browser path: C:\Program Files\Google\Chrome\Application\chrome.exe', 'SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".', 'SLF4J: Defaulting to no-operation (NOP) logger implementation', 'SLF4J: See <http://www.slf4j.org/codes.html#StaticLoggerBinder> for further details.', and 'Jun 02, 2024 10:04:01 PM org.openqa.selenium.devtools.CdpVersionFinder.findNearestMatch'.

Test Results	Duration
Test Results	10 sec 77 ms
Cucumber	10 sec 77 ms
User Login	10 sec 77 ms
Successful login with valid credentials	10 sec 77 ms
User is on the login page	4 sec 786 ms
User submit valid credentials	3 sec 758 ms
User should be redirected to the dashboard	1 sec 141 ms
After(closeBrowser)	392 ms

Command Log

```
✓ Done: Scenarios 1 of 1 (11 sec 478 ms) ⚠
"C:\Program Files\Java\jdk-22\bin\java.exe" ...
Testing started at 10:03 PM ...
Jun 02, 2024 10:03:59 PM org.openqa.selenium.manager.SeleniumManager.lambda$runCommand$1
INFO: Driver path: C:\Users\MSI THIN\.cache\selenium\chromedriver\win64\125.0.6422.141\chromedriver.exe
Jun 02, 2024 10:03:59 PM org.openqa.selenium.manager.SeleniumManager.lambda$runCommand$1
INFO: Browser path: C:\Program Files\Google\Chrome\Application\chrome.exe
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Jun 02, 2024 10:04:01 PM org.openqa.selenium.devtools.CdpVersionFinder.findNearestMatch
```



03

Best Practice



1. Write declarative features

- **Imperative testing** or programming is essentially spelling out with as **much detail** as necessary how to accomplish something.
- **Declarative testing** or programming is only specifying (or declaring) **what needs to be accomplished**.



1. Write declarative features (cont.)

- 
1. Given I open a browser
 2. And I navigate to <http://example.com/login>
 3. When I type in the username field bob97
 4. And I type in the password field F1d0
 5. And I click on **Submit** button
 6. Then I should see the message Welcome Back Bob

Bad sample

- 
1. Given I am on the Login Page
 2. When I sign in with correct credentials
 3. Then I should see a welcome message

Good sample



2. Avoid UI actions steps

- 
1. Given I am on the home page
 2. When I fill in "Username" with: "jondkinney"
 3. And I fill in "Password" with: "SuperSecret123"
 4. And I check "Remember me"
 5. And I press "Log in"
 6. Then a user session should be persisted
 7. And I should be on my dashboard
 8. And I should see "You have successfully logged in."

Bad sample

- 
1. Given I am on the home page
 2. When I login as an admin
 3. Then I should be on my dashboard
 4. And I should see "You have successfully logged in."

Good sample

3. Avoid Conjunctive Steps

Bad sample

1. Given I am on the home page
 2. When I login as an admin
 3. Then I should be on my dashboard and I should see "You have successfully logged in."
- 

Good sample

1. Given I am on the home page
 2. When I login as an admin
 3. Then I should be on my dashboard
 4. And I should see "You have successfully logged in."
- 

4. Limit the number of scenarios per feature

1. Limit one feature per feature file
2. Limit the number of scenarios per feature
3. If feature has too large scope, should be splitted



Q & A

What's Next

- Project Discussion
- Integrate POM for End-to-End Test

Reference

- <https://cucumber.io/docs/guides/overview/>
- <https://www.browserstack.com/guide/automation-using-cucumber-selenium>
- <https://www.linkedin.com/pulse/bdd-cucumber-features-best-practices-liraz-shay/>



Thanks!

Do you have any questions?

shinta.nuraisya@gmail.com



CREDITS: This template has been created by **Slidesgo**, and includes icons by **Flaticon**, infographics & images by **Freepik** and content by **Sandra Medina**

Please keep this slide for attribution



Proyek Akhir

- Mulai minggu depan, silakan **update perkembangan pengerjaan** dengan mengumpulkan **laporan perkembangan**
- Jika ada pertanyaan atau diskusi, silakan untuk diskusi per kelompok ke praktisi, dosen pengampu atau asisten

Tugas

Lakukan implementasi **Gherkin** syntax pada kasus uji proyek di kelas proyek aplikasi dasar menggunakan **Cucumber dan Selenium!**